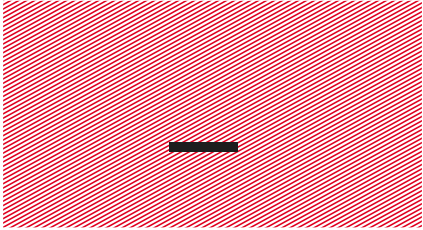


— IoT Devices

Les 3 – Soorten IO-transfers

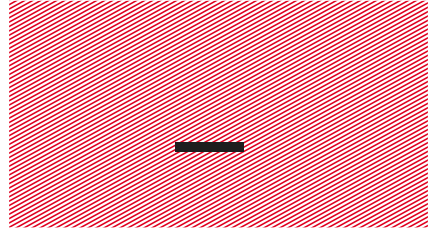




IO-transfers... Huh?



- Wanneer we willen communiceren met de buitenwereld (UART, SPI, I²C, knoppen, ...), kunnen we dat op drie manieren doen:
 - via **polling**.
 - via **interrupts**.
 - via Direct Memory Access (**DMA**).
- De eerste twee, polling en interrupts, werden eerder reeds behandeld in het vak 'Microcontrollers'.

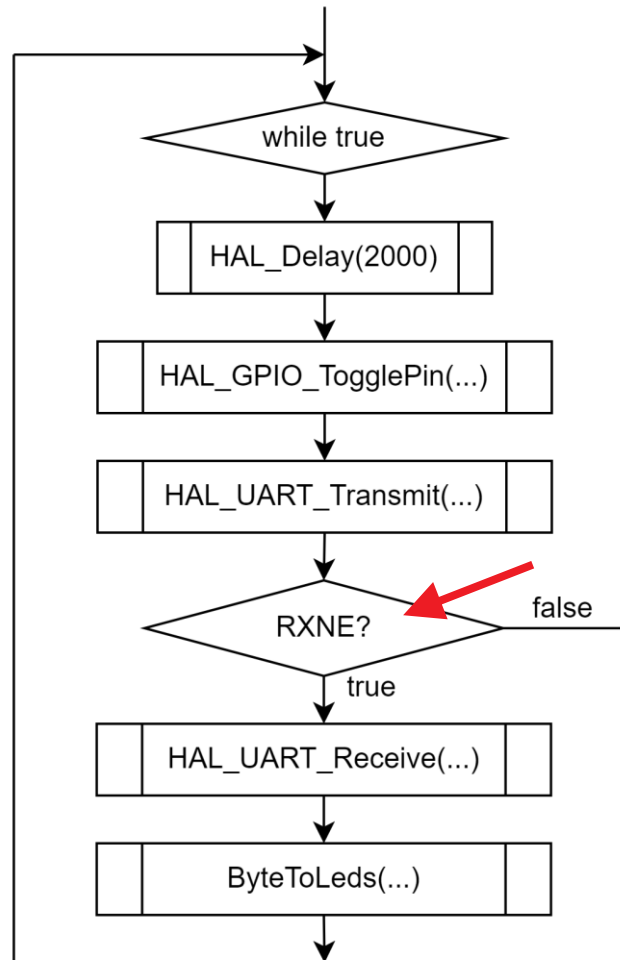


Polling



Polling

Polling = de CPU moet **telkens opnieuw navragen** hoe het gesteld is met ...



Polling

Polling = de CPU moet **telkens opnieuw navragen** hoe het gesteld is met ...

```
106 while (1)
107 {
108     // Even wachten.
109     HAL_Delay(2000);
110
111     // User LED toggle'n.
112     HAL_GPIO_TogglePin(UserLED_GPIO_Port, UserLED_Pin);
113
114     // Teken van leven geven.
115     HAL_UART_Transmit(&huart2, (uint8_t*)MESSAGE, strlen(MESSAGE), HAL_MAX_DELAY);
116
117     // Is er een een byte ontvangen via de USART2?
118     // Lees de info in via 'polling' (= op gezette momenten vragen of er info is).
119     if(__HAL_UART_GET_FLAG(&huart2, UART_FLAG_RXNE))
120     {
121         HAL_UART_Receive(&huart2, (uint8_t*)&receivedData, 1, HAL_MAX_DELAY);
122         ByteToLeds(receivedData);
123     }
```

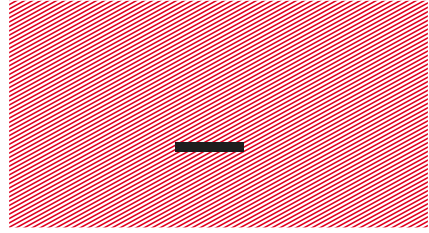
Bekijk zeker de werking van
`HAL_UART_Transmit()`
en
`HAL_UART_Receive()`.

Polling

Polling = de CPU moet **telkens opnieuw navragen** hoe het gesteld is met ...

```
106 while (1)
107 {
108     // Even wachten.
109     HAL_Delay(2000);
110
111     // User LED toggle'n.
112     HAL_GPIO_TogglePin(UserLED_GPIO_Port, UserLED_Pin);
113
114     // Teken van leven geven.
115     HAL_UART_Transmit(&huart2, (uint8_t*)MESSAGE, strlen(MESSAGE), HAL_MAX_DELAY);
116
117     // Is er een byte ontvangen via de USART2?
118     // Lees de info in via 'polling' (= op gezette momenten vragen of er info is).
119     if(__HAL_UART_GET_FLAG(&huart2, UART_FLAG_RXNE))
120     {
121         HAL_UART_Receive(&huart2, (uint8_t*)&receivedData, 1, HAL_MAX_DELAY);
122         ByteToLeds(receivedData);
123     }
```

Als er een zekere tijd verloopt tussen de momenten waarop je een 'vlag' controleert, **kan het zijn dat je data mist**. Dat is mogelijks rampzalig!

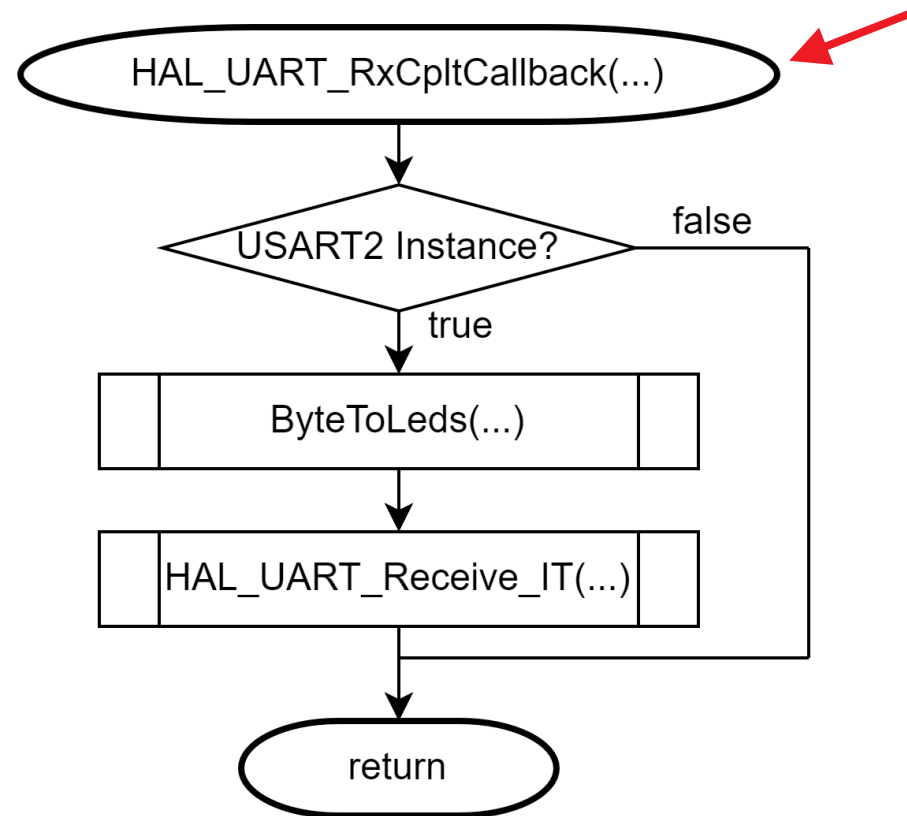
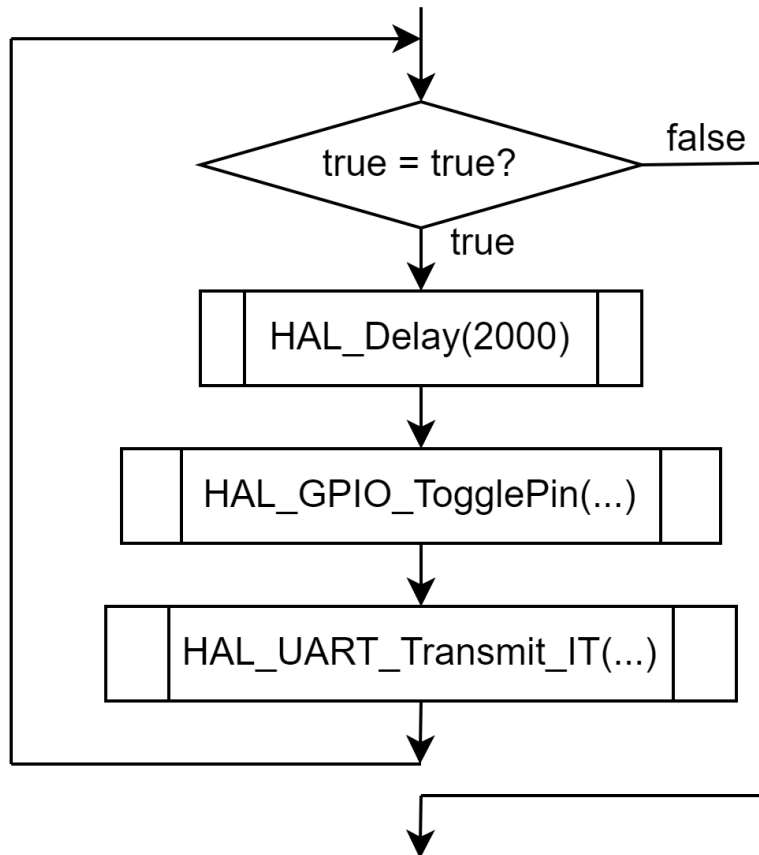


Interrupts



Interrupts

Interrupts = de CPU **wordt verwittigd** van een bepaalde situatie via een interruptvlag...



Interrupts

Interrupts = de CPU **wordt verwittigd** van een bepaalde situatie via een interruptvlag...

```
109 while (1)
110 {
111     // Even wachten.
112     HAL_Delay(2000);
113
114     // User LED toggle'n.
115     HAL_GPIO_TogglePin(UserLED_GPIO_Port, UserLED_Pin);
116
117     // Teken van leven geven.
118     HAL_UART_Transmit_IT(&huart2, (uint8_t*)MESSAGE, strlen(MESSAGE));
```

De hoofdloop bevat enkel nog code om UART-data te verzenden. Het ontvangen gebeurt elders... In de interrupt subroutine...

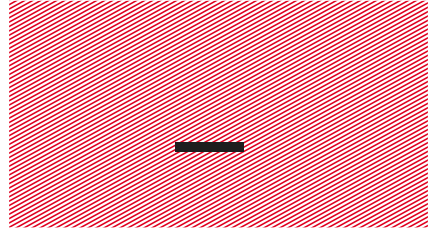
Interrupts

Interrupts = de CPU **wordt verwittigd** van een bepaalde situatie via een interruptvlag...

```
358 // Code hieronder wordt opgeroepen als alle data van HAL_UART_Receive_IT() ontvangen is...
359 // Deze functie wordt opgeroepen vanuit HAL_UART_IRQHandler().
360 void HAL_UART_RxCpltCallback(UART_HandleTypeDef *UartHandle)
361 {
362     // Is het data van UART2?
363     if(UartHandle->Instance == USART2)
364     {
365         // Ontvangen data op de LED's zetten.
366         ByteToLeds(receivedData);
367
368         // Ontvangst via interrupts opnieuw starten. Want RXNEIE wordt automatisch uitgeschakeld.
369         HAL_UART_Receive_IT(&huart2, (uint8_t*)&receivedData, 1);
370     }
371 }
```

De interrupt code kan focussen op de ontvangst van data van de UART. Daardoor is de **kans minimaal dat je data misloopt!** Het nadeel is wel dat de CPU moet 'wegspringen' van de hoofdloop naar de interrupt code.

Houd interrupt code **zo kort mogelijk** (qua tijdsbestek)!



Direct Memory Access





Direct Memory Access

Direct Memory Access (DMA) = de CPU **wordt vrijgesteld** van het afhandelen van de communicatie. Dat doet de DMA controller...

Onze microcontroller heeft twee DMA controllers. Dat zijn **systemen die onafhankelijk van de CPU**, ook gegevens kunnen uitwisselen tussen 'peripherals' en 'memories'.

We kunnen dus bijvoorbeeld instellen dat er tien bytes opgevraagd worden van UART2 en naar het geheugen gekopieerd worden, **ZONDER dat de CPU daarvoor nodig is**. Whoehoe!



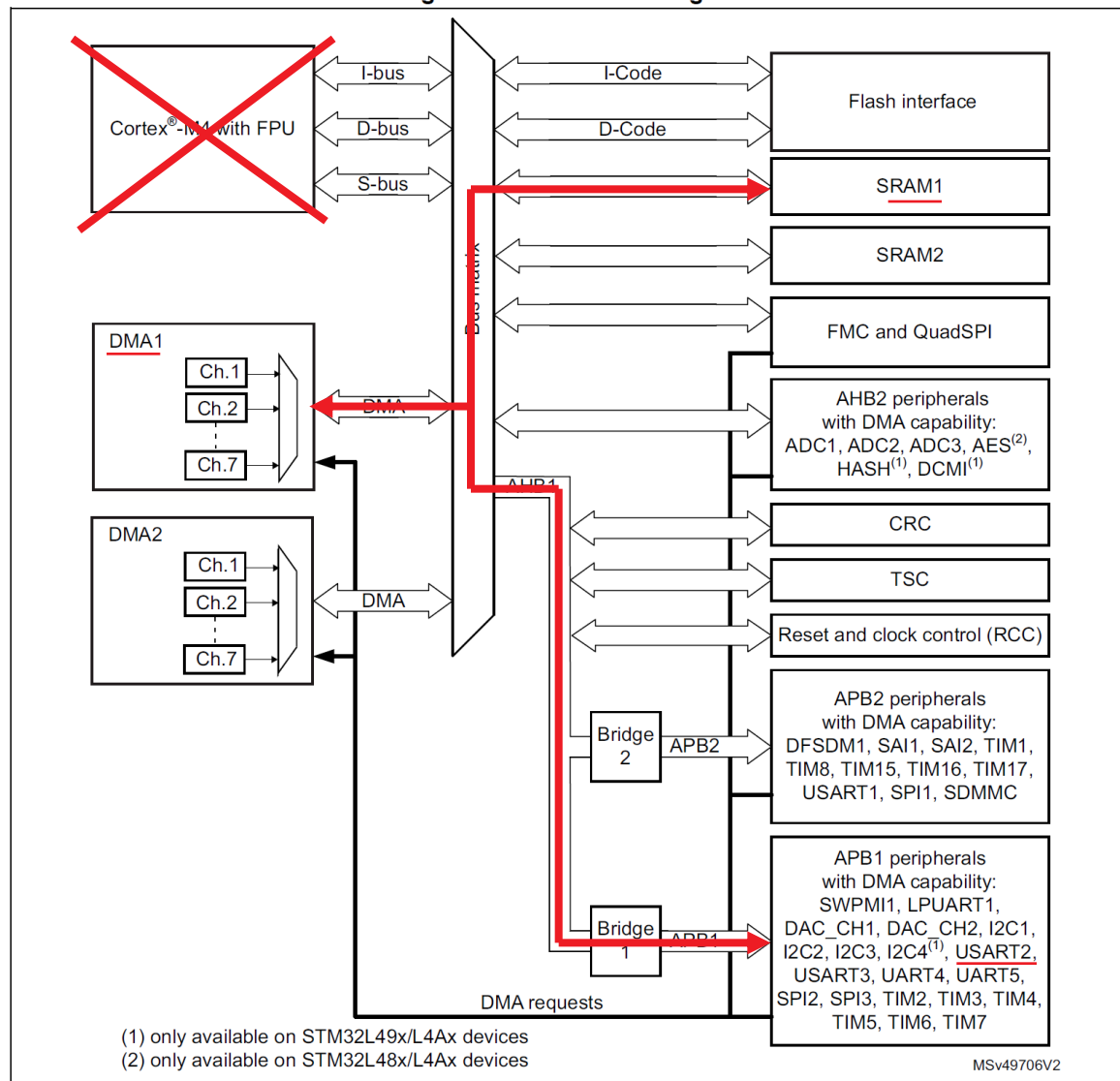
Direct Memory Access

Merk op: aan de linker kant zie je de CPU, DMA1 en DMA2. Dat zijn drie 'bus masters'.

Ze hebben toegang tot de 'Bus Matrix' en kunnen **zelfstandig data uitwisselen**.

Er is wel **nog één interrupt nodig** om aan te geven dat de uitwisseling voorbij is...

Figure 31. DMA block diagram



Bron: RM0351 ST Reference Manual.

Direct Memory Access

Zowel de CPU als de twee DMA controllers kunnen de **systeembussen gebruiken**. Het is de bus matrix die het 'round-robin' schema implementeert zodat alles mooi gestroomlijnd geraakt. Iedere deelnemer krijgt dus gelijkwaardige toegang tot de gedeelde bussen.

The DMA controller performs direct memory transfer by sharing the AHB system bus with other system masters. The bus matrix implements round-robin scheduling. DMA requests may stop the CPU access to the system bus for a number of bus cycles, when CPU and DMA target the same destination (memory or peripheral).

Bron: RM0351 ST Reference Manual. (11.4.1 DMA)

Zie ook: [https://nl.wikipedia.org/wiki/Round-robin_\(informatietechnologie\)](https://nl.wikipedia.org/wiki/Round-robin_(informatietechnologie))



Direct Memory Access

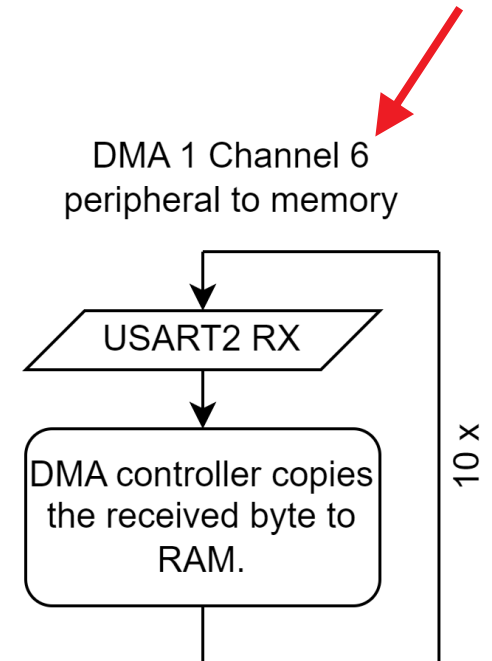
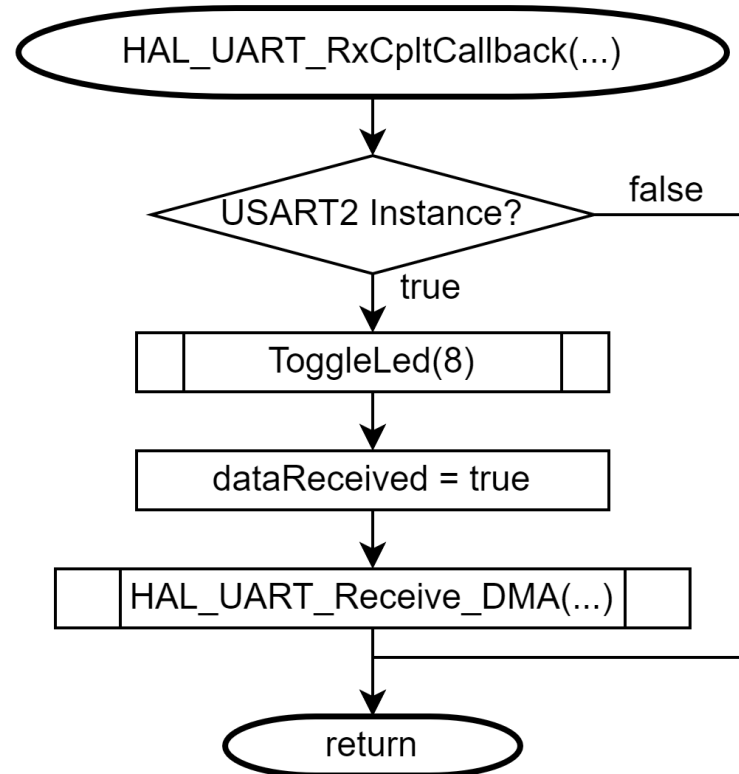
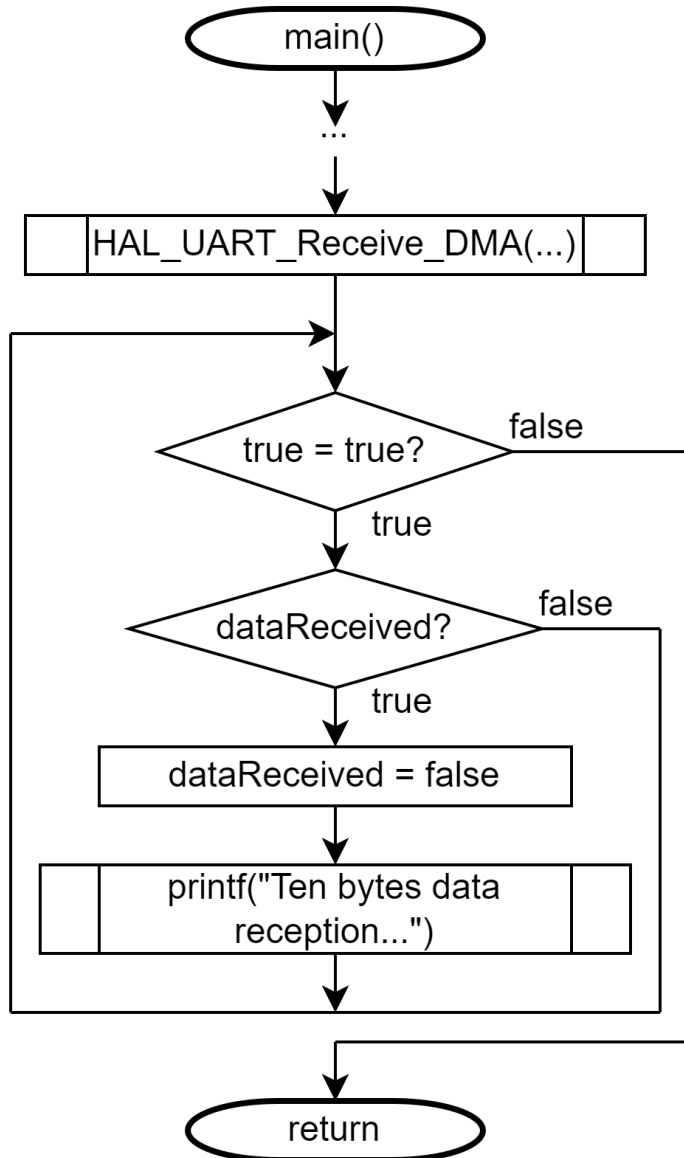
Indien verschillende kanalen van één bepaalde DMA Controller gebruikt worden, kan je de **prioriteit** instellen via software en is er ook een hardware prioriteit indien nodig.

Priority between the requests is programmable by software (4 levels per channel: very high, high, medium, low) and by hardware in case of equality (such as request to channel 1 has priority over request to channel 2).

Bron: RM0351 ST Reference Manual. (11.2 DMA)



Direct Memory Access



Age Group	Percentage
18-24	10
25-34	15
35-44	20
45-54	25
55-64	30
65-74	35
75-84	40
85+	45



Direct Memory Access

STM32CubeMX CubeMX - Template - Nucleo-L476RG.ioc: STM32L476RGTx NUCLEO-L476RG

File Window Help myST

Home > STM32L476RGTx - NUCLEO-L476RG > CubeMX - Template - Nucleo-L476RG.ioc - Pinout & Configuration

GENERATE CODE

Pinout & Configuration Clock Configuration Project Manager Tools

Software Packs Pinout

USART2 Mode and Configuration

Mode

Mode Asynchronous

Hardware Flow Control (RS232) Disable

Hardware Flow Control (RS485)

Configuration

Reset Configuration

Parameter Settings User Constants NVIC Settings DMA Settings GPIO Settings

NVIC Interrupt Table	Enable	Preemption Priority	Sub Priority
DMA1 channel6 global interrupt	☒	0	0
USART2 global interrupt	☐	0	0

Pinout view System view

STM32L476RGTx LQFP64

Categories A-Z

WWDG

Analog >

Timers >

Connectivity >

CAN1

I2C1

I2C2

I2C3

IRTIM

LPUART1

QUADSPI

SDMMC1

SPI1

SPI2

SPI3

SWPMI1

UART4

UART5

USART1

USART2

USART3

USB_OTG_FS

Multimedia >

VBAT..

PC13..

PC14..

PC15..

PH0..

PH1..

NRST

LED1

SW4

PC0

PC1

PC2

PC3

VSS..

POT

PA0

SW1

PA1

TX2

PA2

PA3

VSS

VDD

PA4

PA5

PA6

PA7

PC4

PC5

PB0

PB1

PB2

PB10

PB11

VSS

VDD

LED5

UserLED

SW2

SW3

LED8

LED3

LED4

LED2

PC12

PC11

PC10

PA15..

PA14..

TCK

VDD..

VSS

PA13..

PA12

PA11

PA10

PA9

PA8

LED6

PC9

PC8

LED7

PC6

PB15

PB14

PB13

PB12

TMS

Direct Memory Access

```
107 // Ontvangst van 10 bytes via DMA starten.
108 HAL_UART_Receive_DMA(&huart2, destination, NUMBER_OF_BYTES_TO_RECEIVE);
109
110 /* USER CODE END 2 */
111
112 /* Infinite loop */
113 /* USER CODE BEGIN WHILE */
114 while (1)
115 {
116     // Is er data ontvangen via DMA?
117     if(dataReceived)
118     {
119         // Melding wissen.
120         dataReceived = false;
121
122         // Gebruiker informeren (niet via DMA).
123         printf("Ten bytes data reception complete...\r\n");
124     }
```



CubeMX - HAL - Nucleo-L476RG - DMA - UART to memory

Direct Memory Access

```
380 // De code hieronder wordt opgeroepen als alle data van HAL_UART_Receive_DMA() ontvangen is...
381 // Deze functie wordt opgeroepen vanuit:
382 //     DMA1_Channel6_IRQHandler() -> HAL_DMA_IRQHandler() -> HAL_UART_RxCpltCallback().
383 void HAL_UART_RxCpltCallback(UART_HandleTypeDef *UartHandle)
384 {
385     // Is het data van UART2?
386     if(UartHandle->Instance == USART2)
387     {
388         ToggleLed(8);                // Teken van leven geven...
389         dataReceived = true;         // Melden aan de hoofdlus.
390
391         // Ontvangst via DMA opnieuw starten. Want in het DMA1_Channel6->CCR register,
392         // wordt de TCIE bit automatisch uitgeschakeld.
393         // Dat gebeurt in: HAL_DMA_IRQHandler().
394         HAL_UART_Receive_DMA(&huart2, destination, NUMBER_OF_BYTES_TO_RECEIVE);
395     }
396 }
```

Direct Memory Access

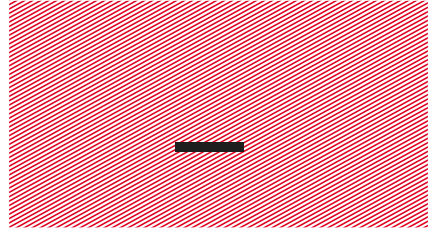
Tot slot:

- Je kan ook DMA gebruiken voor het **verzenden** van data.
- Er zijn **vele peripherals** die DMA-ondersteuning hebben: UART, SPI, I²C, AD, DA, Timers, ...
- Zoek steeds op **welk 'channel'** van de DMA controller je kan gebruiken voor welke peripheral.

Table 44. DMA1 requests for each channel

CxS[3:0]	Channel 1	Channel 2	Channel 3	Channel 4	Channel 5	Channel 6	Channel 7
0000	ADC1	ADC2	ADC3	DFSDM1_FLT0	DFSDM1_FLT1	DFSDM1_FLT2	DFSDM1_FLT3
0001	-	SPI1_RX	SPI1_TX	SPI2_RX	SPI2_TX	SAI2_A	SAI2_B
0010	-	USART3_TX	USART3_RX	USART1_TX	USART1_RX	USART2_RX	USART2_TX

Bron: RM0351 ST Reference Manual.



Interrupt van een knop





Interrupt van een knop

- Je kan uiteraard ook met **HAL-bibliotheken** een **interrupt** van een **knop** binnenlezen.
- Daarvoor gebruik je de microcontroller pinnen met **EXTI-mogelijkheden**.
- Zie module 'Microcontrollers' van in fase 1 voor CMSIS-voorbeelden.
- Zorg ervoor dat in STM32CubeMX bij **NVIC** (Nested Vectored Interrupt Controller), de juiste EXTI line x interrupt enable **aangevinkt** staat.

Interrupt van een knop

STM32CubeMX CubeMX - Template - Nucleo-L476RG.ioc*: STM32L476RGTx NUCLEO-L476RG

File Window Help myST

Home STM32L476RGTx - NUCLEO-L476RG CubeMX - Template - Nucleo-L476RG.ioc - Pinout & Configuration GENERATE CODE

Pinout & Configuration Clock Configuration Project Manager Tools

Software Packs Pinout

GPIO Mode and Configuration

Configuration

Group By Peripherals

GPIO ADC SYS USART NVIC

Search Signals Search (Ctrl+F) ☐ Show only Modified Pins

Pin Name	Sign...	GPIO...	GPIO mode	GPI...	Max...	Fas...	Use...	Mod...
PA1	n/a	n/a	External Interrupt Mode with Falling edge trigger detection	No ...	n/a	n/a	SW1	☒
PA4	n/a	n/a	Input mode	No ...	n/a	n/a	SW2	☒
PB0	n/a	n/a	Input mode	No ...	n/a	n/a	SW3	☒
PC1	n/a	n/a	Input mode	No ...	n/a	n/a	SW4	☒
PC13	n/a	n/a	Input mode	No ...	n/a	n/a	Use...	☒
PA5	n/a	Low	Output Push Pull	No ...	Low	n/a	Use...	☒
PA8	n/a	Low	Output Push Pull	No ...	Low	n/a	LED6	☒
PB3 (JTDO...	n/a	Low	Output Push Pull	No ...	Low	n/a	LED2	☒
PB4 (NJTR...	n/a	Low	Output Push Pull	No ...	Low	n/a	LED4	☒
PB5	n/a	Low	Output Push Pull	No ...	Low	n/a	LED3	☒

Select Pins from table to configure them. Multiple selection is Allowed.

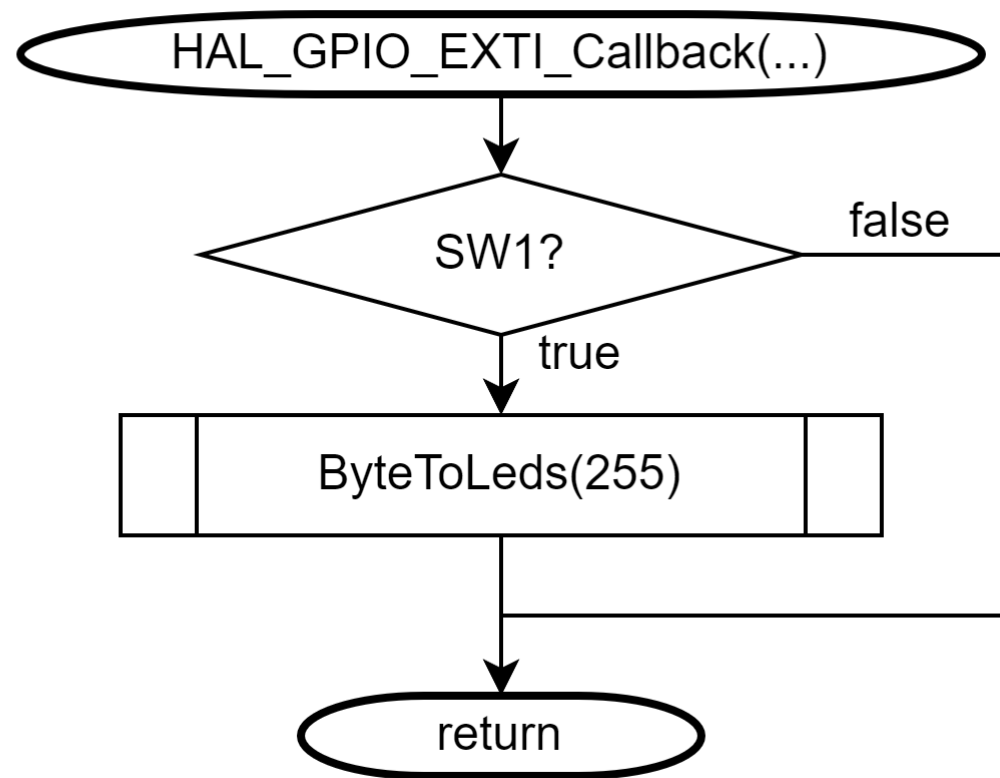
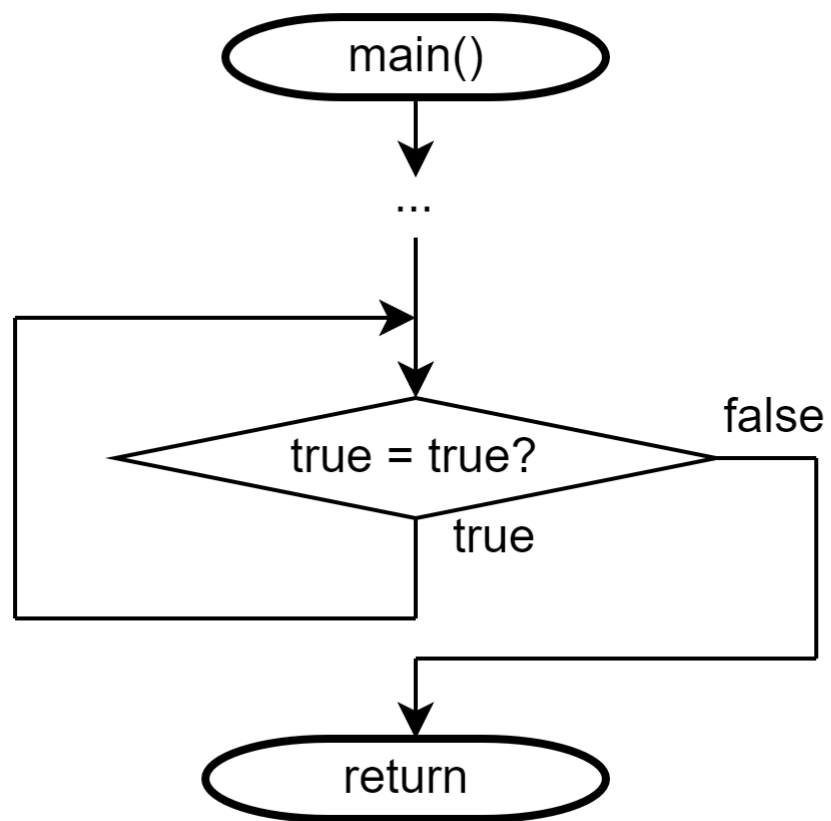
Pinout view System view

STM32L476RGTx LQFP64

UserButton LED1 SW4 LED6 LED7

Demo: Oplossing oefening 11 - interrupt van een knop

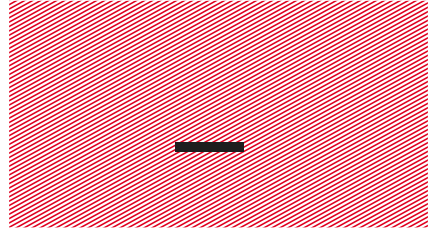
Interrupt van een knop



Interrupt van een knop

```
352 // Als er een interrupt is, dan wordt de EXTI1_IRQHandler()  
353 // interrupthandler uitgevoerd in 'stm32l4xx_it.c'.  
354 // In die functie, wordt de interruptvlag gereset en onderstaande  
355 // callback functie opgeroepen...  
356 // De instellingen om de interrupt code te maken, wordt gedaan in  
357 // STM32CubeMX bij de juiste pin.  
358 void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)  
359 {  
360     // Werd er op SW1 gedrukt?  
361     if(GPIO_Pin == SW1_Pin)  
362         ByteToLeds(255);  
363 }
```

Houd ook hier (zoals ALTIJD) de interrupt code **zo kort mogelijk** (qua tijdsbestek)!



Vergelijking: UART-data verzenden via polling en DMA...



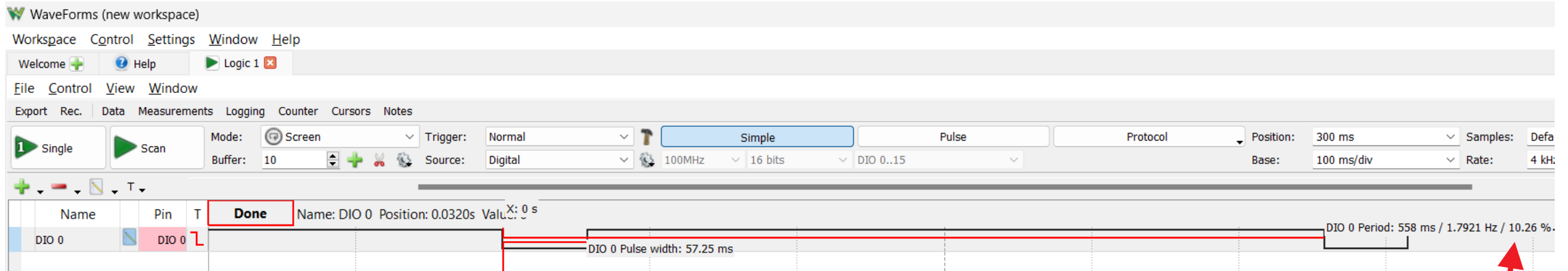
Vergelijking: UART TX **polling** versus DMA

Polling

```
106 while (1)
107 {
108     // LED1 aanzetten. Dit geeft aan dat de hoofdlus begint met nuttige zaken...
109     // Breng de toestand van LED1 in beeld met een USB oscilloscoop.
110     HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin, GPIO_PIN_SET);
111
112     // Hier kan dan nuttige code komen...
113
114     // LED1 uitzetten. Dit geeft aan dat de hoofdlus begint met verzenden van
115     // de UART-data...
116     HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin, GPIO_PIN_RESET);
117
118     // UART-data verzenden starten (in een blokkerende functie).
119     HAL_UART_Transmit(&huart2, (uint8_t*)MESSAGE, strlen(MESSAGE), HAL_MAX_DELAY);
120
121     // LED1 aanzetten. Dit geeft aan dat de hoofdlus begint met nuttige zaken...
122     HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin, GPIO_PIN_SET);
123
124     // Hier kan ook nuttige code komen...
125
126     // Even wachten. Het wachten is dikwijls geen goed idee, maar
127     // hier helpt het om de zaken beter te begrijpen (hopelijk) ;-).
128     HAL_Delay(500);
129     /* USER CODE END WHILE */
130 }
```

Vergelijking: UART TX **polling** versus DMA

Polling



Van de tijd die de hoofdlus ter beschikking heeft, gaat er dus meer dan **10% 'verloren'** aan het versturen van de UART-data. Dat is meer dan 57ms!

Die tijd kan wellicht nuttiger besteed worden door de CPU....

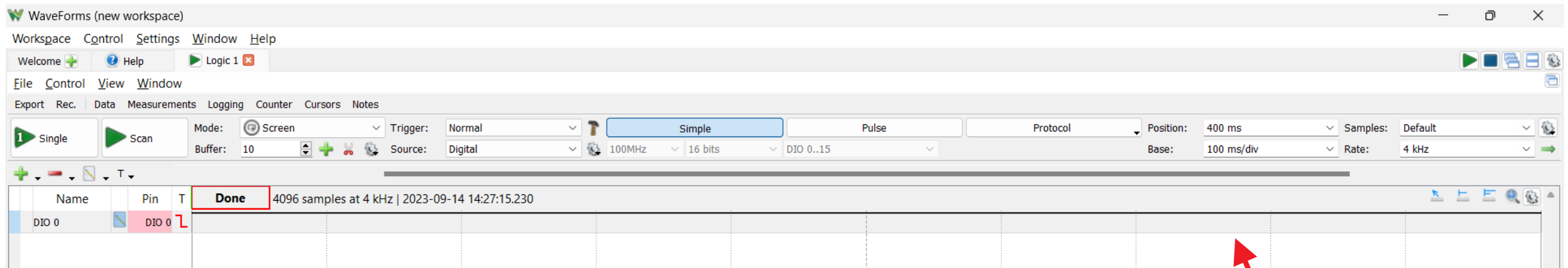
Vergelijking: UART TX polling versus **DMA**

DMA

```
110 while (1)
111 {
112     // LED1 aanzetten. Dit geeft aan dat de hoofdlus begint met nuttige zaken...
113     // Breng de toestand van LED1 in beeld met een USB oscilloscoop.
114     HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin, GPIO_PIN_SET);
115
116     // Hier kan dan nuttige code komen...
117
118     // LED1 uitzetten. Dit geeft aan dat de hoofdlus begint met verzenden van
119     // de UART-data...
120     HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin, GPIO_PIN_RESET);
121
122     // UART-data verzenden starten (met DMA instellingen).
123     HAL_UART_Transmit_DMA(&huart2, (uint8_t*)MESSAGE, strlen(MESSAGE));
124
125     // LED1 aanzetten. Dit geeft aan dat de hoofdlus begint met nuttige zaken...
126     HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin, GPIO_PIN_SET);
127
128     // Hier kan ook nuttige code komen...
129
130     // Even wachten. Het wachten is dikwijls geen goed idee, maar
131     // hier helpt het om de zaken beter te begrijpen (hopelijk ;-).
132     HAL_Delay(500);
```

Vergelijking: UART TX polling versus **DMA**

DMA

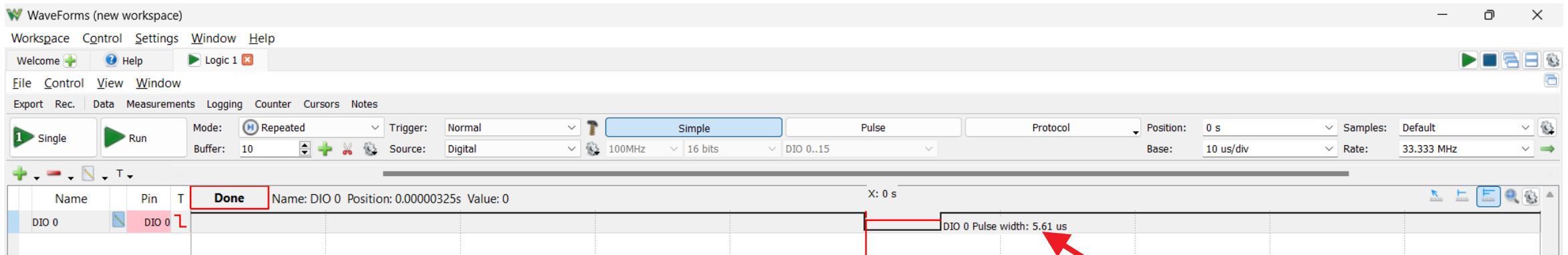


De tijd waarin de CPU nu onbeschikbaar is, is zodanig kort dat hij bij een grote tijdsbasis niet zichtbaar is op de oscilloscoop ...

De CPU heeft dus **quasi het volledige tijdsbestek beschikbaar** om nuttige berekeningen te doen....

Vergelijking: UART TX polling versus **DMA**

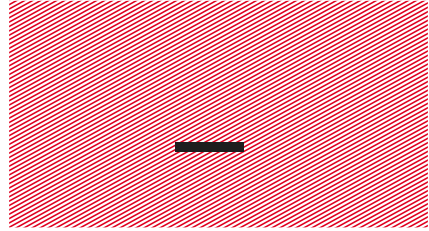
DMA



Als we toch wat inzoomen, zien we dat er wel degelijk iets van 'tijdverlies' is. Maar dat is ongelooflijk klein: 5,61µs.

Dat is dus **meer dan 10 000 keer minder tijdsverlies** dan de code met polling!

#leveDMA



Demo: SBUS inlezen via DMA

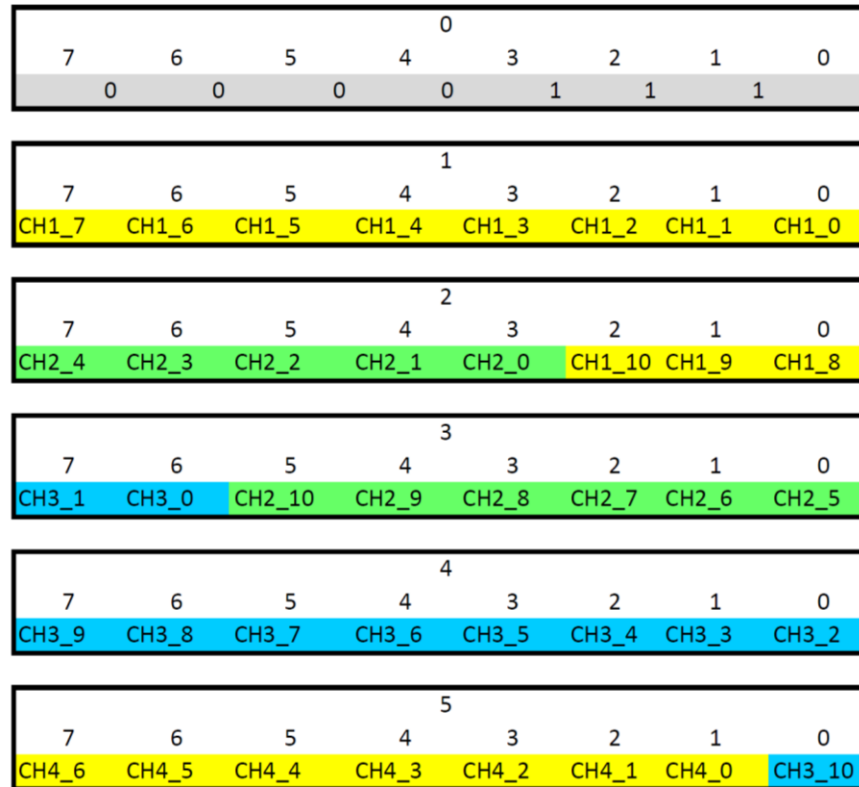


- **SBUS** = veel gebruikte bus om bij drones en andere RC-toepassingen te communiceren tussen de handzender en het telegeleid toestel.
- Is in feite een seriële bus (**UART**), met als eigenschappen:
 - 100k baud.
 - two stopbits.
 - even parity.
 - RX active level inverted.
- Weet je? Je kan synchroniseren door de **Receiver Timeout** van de UART goed in te stellen...



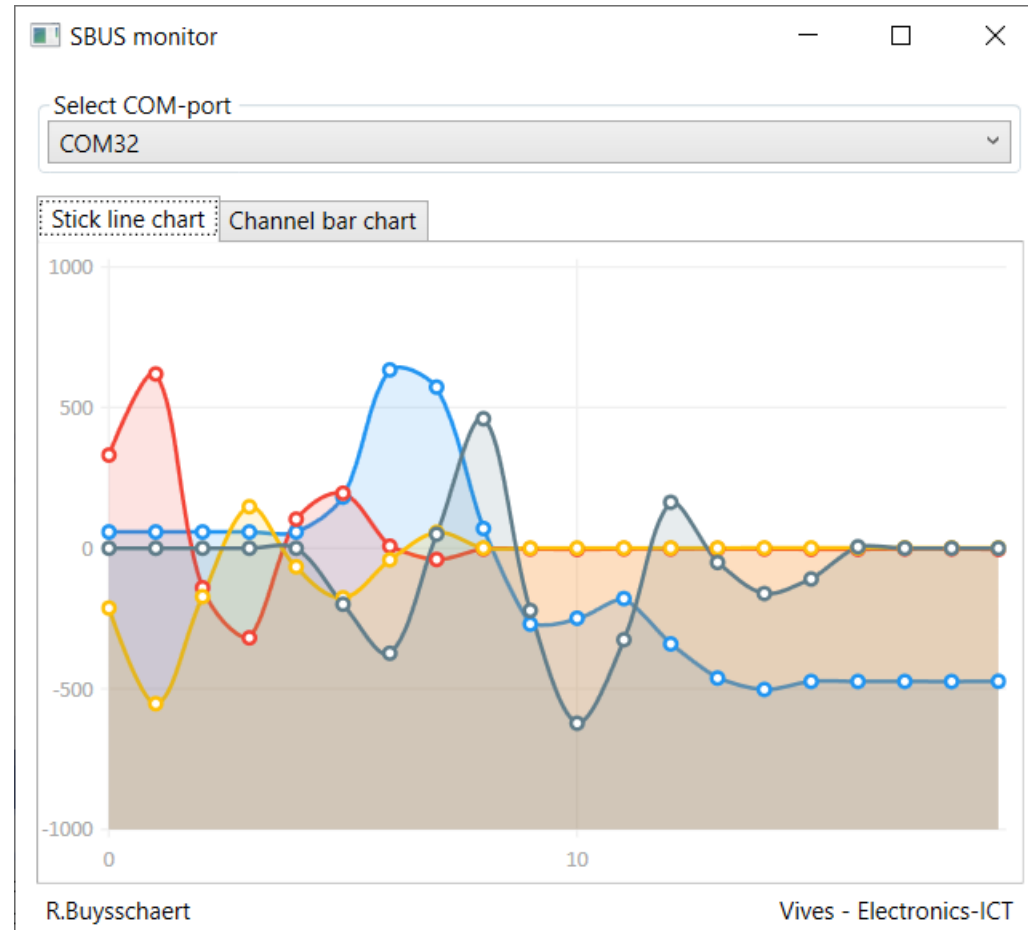
SBUS inlezen via DMA

De SBUS '**kanalen**', bestaan uit 11-bit waarden. Daarom moet je verschillende schuifoperaties doen om de datastroom om te zetten naar integer getallen.



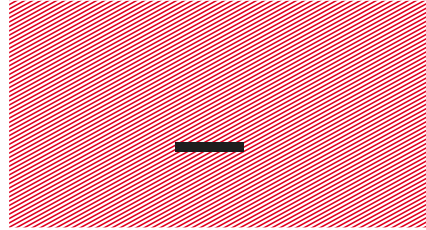
SBUS inlezen via DMA

- **Demo:** met SBUS en APA102C LED's.



SBUS inlezen via DMA

```
514 // Code hieronder wordt opgeroepen als alle data van HAL_UART_Receive_DMA() ontvangen is...
515 // Deze functie wordt opgeroepen vanuit HAL_UART_IRQHandler().
516 void HAL_UART_RxCpltCallback(UART_HandleTypeDef *UartHandle)
517 {
518     // Is het data van UART1?
519     if(UartHandle->Instance == USART1)
520     {
521         // In functie van scoopmetingen, LED8 omkeren.
522         HAL_GPIO_TogglePin(LED8_GPIO_Port, LED8_Pin);
523
524         // LED1 inschakelen (i.f.v. tijdsregistratie met scoop).
525         HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin, GPIO_PIN_SET);
526
527         // SBUS-data proberen omzetten naar kanaalwaardes.
528         if(SBUSConvert(&sbus))
529             sbus.newFrameConverted = true;
530
531         // LED1 uitschakelen (tijdsregistratie).
532         HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin, GPIO_PIN_RESET);
533     }
534 }
535
536 // Is er een Receiver Timeout geweest (hier behandeld als error)? Hersynchroniseer.
537 // Let op: deze functie wordt enkel opgeroepen als 'UART1 Global Interrupt' aangevinkt staat in CubeMX!
538 void HAL_UART_ErrorCallback(UART_HandleTypeDef *huart)
539 {
540     if((huart->ErrorCode & HAL_UART_ERROR_RTO) == HAL_UART_ERROR_RTO)
541     {
542         // In functie van scoopmetingen, LED7 omkeren.
543         HAL_GPIO_TogglePin(LED7_GPIO_Port, LED7_Pin);
544
545         // Ontvangst via interrupts opnieuw starten. Want RXNEIE wordt automatisch uitgeschakeld.
546         HAL_UART_Receive_DMA(huart, (uint8_t*)sbus.rawData, SBUS_NUMBER_OF_BYTES);
547     }
548 }
```



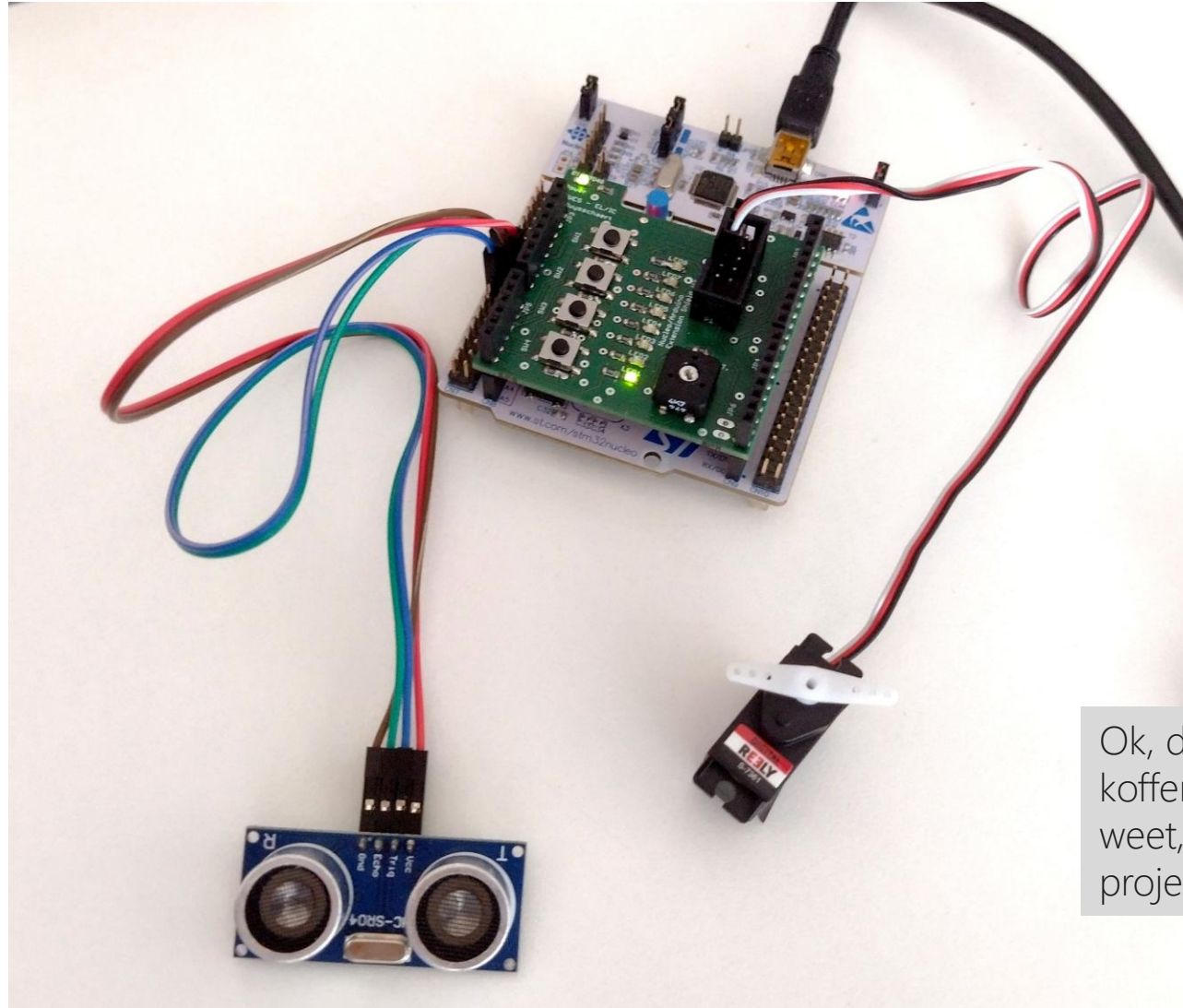
Demo: automatisch kofferdeksel



- Kunnen we een 'automatisch kofferdeksel' simuleren?
- Gebruikte onderdelen:
 - Nucleo-L476RG.
 - HC-SR04 ultrasone sensor.
 - modelbouwservo.
- Gebruikte technieken:
 - Timer Input Capture.
 - Timer PWM.

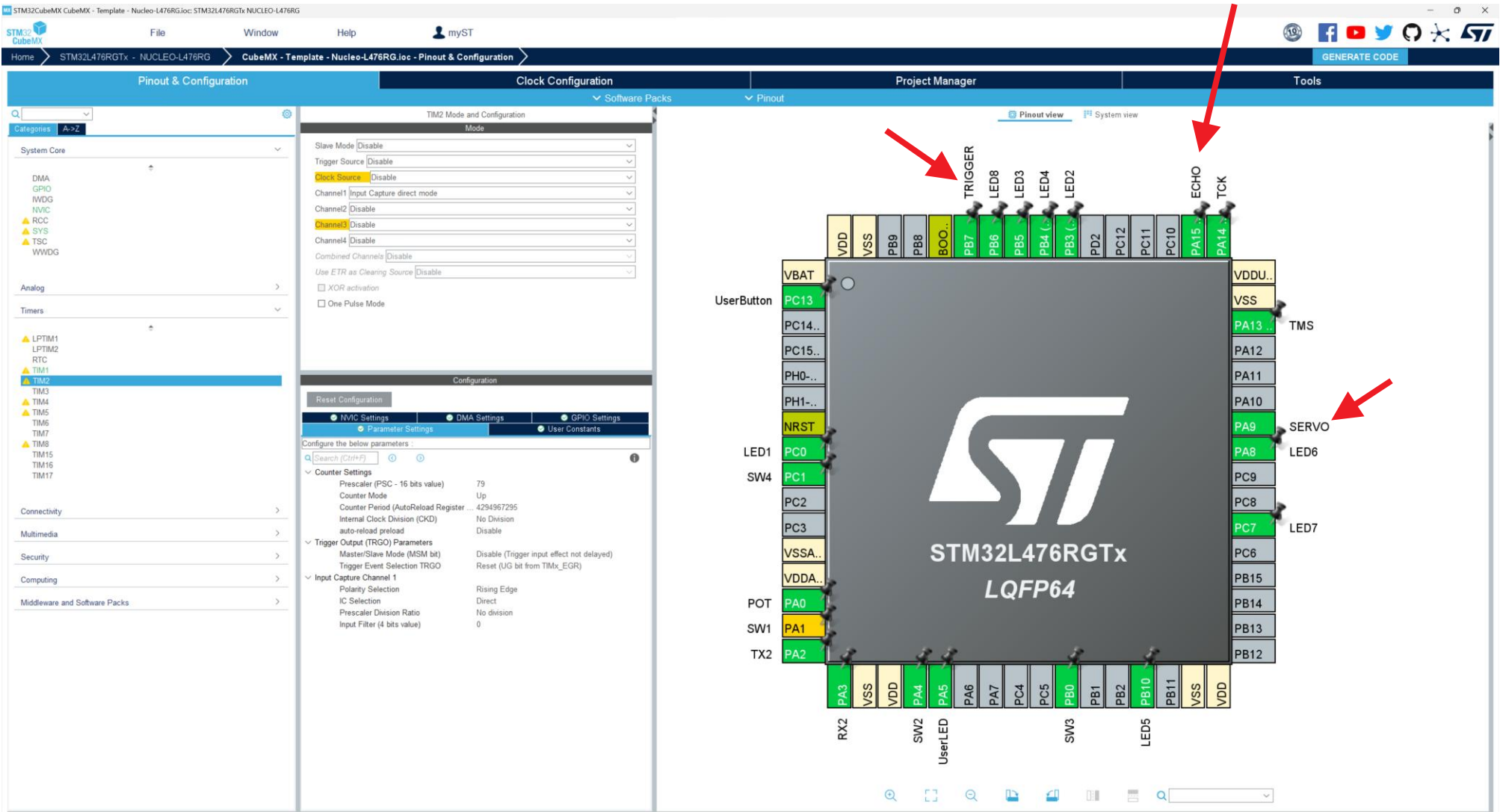


Automatisch kofferdeksel



Ok, dit lijkt nog niet op een kofferdeksel... Maar wie weet, komt er nog een projectje aan....

Automatisch kofferdeksel



CubeMX - HAL - Nucleo-L476RG - Interrupt - Timer 2 - Kofferdeksel HC-SR04 en PWM Servo

Automatisch kofferdeksel

```
143 while (1)
144 {
145     // Trigger uitsturen zodat de meting start.
146     HAL_GPIO_WritePin(TRIGGER_GPIO_Port, TRIGGER_Pin, GPIO_PIN_SET);
147     HAL_Delay(1);
148     HAL_GPIO_WritePin(TRIGGER_GPIO_Port, TRIGGER_Pin, GPIO_PIN_RESET);
149
150     // Even wachten op het onhoorbaar geluid.
151     HAL_Delay(100);
152
153     // De tijd wordt gemeten in microseconden. Bereken daaruit de afstand.
154     // 1cm (moet dubbel afgelegd worden) = 0.02m/340m/s = ~59 µs.
155     // Enkel de waarden boven de 2 cm en onder de 120 cm vertrouwen.
156     distance = capture/ONE_CM_SOUND_TRAVELLING_US_TIME;
157     if((distance >= CM_DISTANCE_MINIMUM_RELIABLE_THRESHOLD) && (distance <= CM_DISTANCE_MAXIMUM_RELIABLE_THRESHOLD))
158     {
159         // JSON-data versturen met daarin de gemeten afstand in centimeters.
160         printf("{\"distance\":%d}\\r\\n", distance);
161         ByteToLevel(distance*2);
162
163         // Kofferdeksel openen door de PWM voor de servo te wijzigen.
164         if(distance < CM_DISTANCE_TO_OPEN_DOOR)
165             __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_2, PWM_OFFSET + PWM_SETPOINT);
166     }
167     else
168     {
169         // Ongeldige afstand gemeten. Verstuur -1 via JSON.
170         printf("{\"distance\":-1}\\r\\n");
171         ByteToLevel(0);
172     }
173
174     // Kofferdeksel terug sluiten wanneer er op de User Button gedrukt wordt.
175     if(UserButtonActive())
176         __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_2, PWM_OFFSET);
```

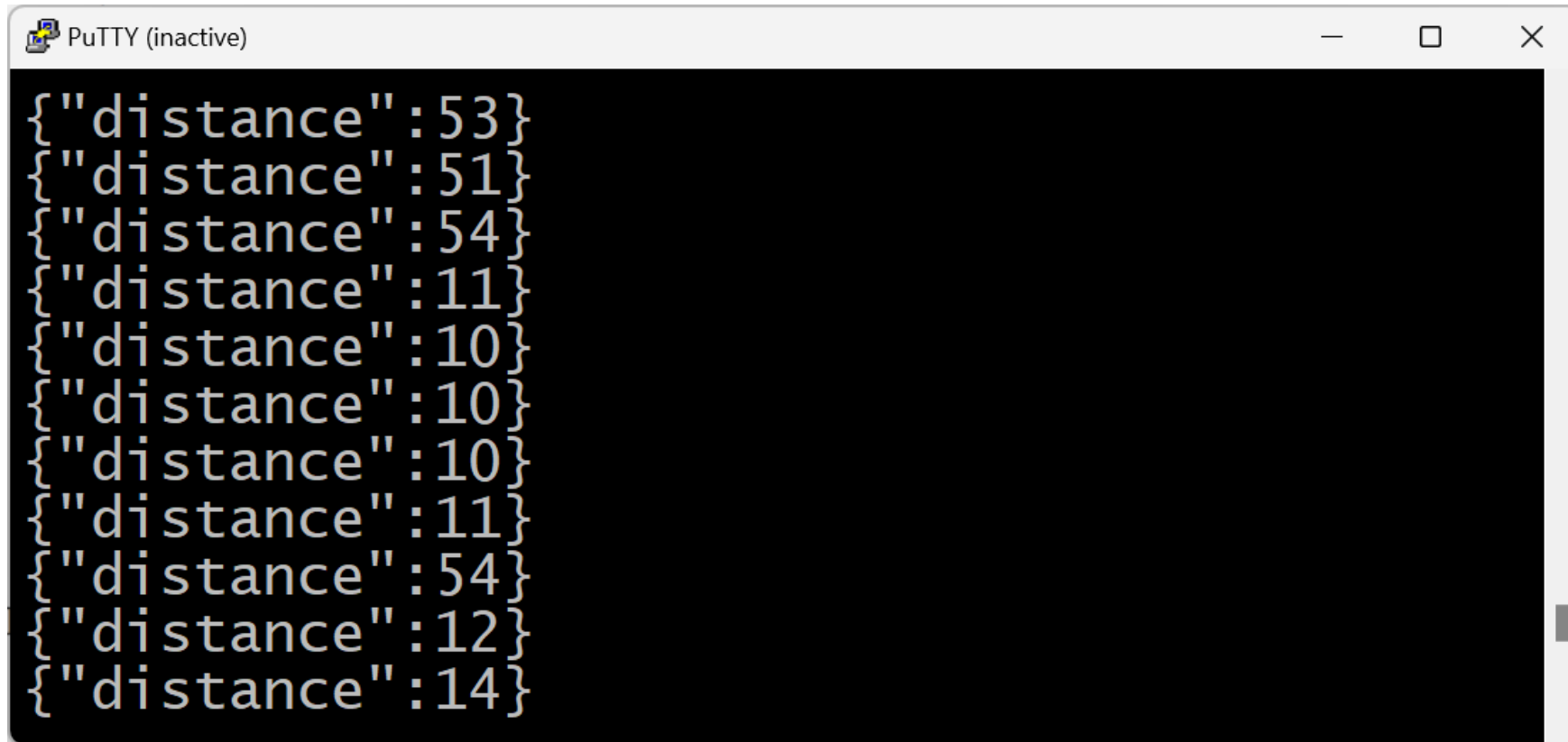
Automatisch kofferdekse

```

525 // Callback voor iedere interrupt van Timer 2 opvangen.
526 void HAL_TIM_IC_CaptureCallback (TIM_HandleTypeDef * htim)
527 {
528     // Is het Timer 2?
529     if(htim->Instance == TIM2)
530     {
531         // Kijken wat de staat van de ECHO-pin is. Afhankelijk daarvan weet je of je een dalende of stijgende flank had.
532         if(HAL_GPIO_ReadPin(ECHO_GPIO_Port, ECHO_Pin) == GPIO_PIN_SET)
533         {
534             // Stijgende flank gezien. Begin van de meting.
535
536             // Interruptvlag resetten.
537             __HAL_TIM_CLEAR_IT(htim, TIM_IT_CC1);
538
539             // Teller resetten.
540             __HAL_TIM_SET_COUNTER(&htim2, 0);
541
542             // Vanaf nu wachten op een falling edge.
543             __HAL_TIM_SET_CAPTUREPOLARITY(htim, TIM_CHANNEL_1, TIM_INPUTCHANNELPOLARITY_FALLING);
544
545             // Visueel aangeven dat de stijgende flank er is.
546             SetUserLed(true);
547         }
548         else
549         {
550             // Dalende flank gezien. Einde van de meting.
551
552             // Gemeten tijd opslaan. Hierdoor wordt ook de interruptvlag gereset.
553             capture = __HAL_TIM_GET_COMPARE(htim, TIM_CHANNEL_1);
554
555             // Visueel aangeven dat de dalende flank er is.
556             SetUserLed(false);
557
558             // Wachten op rising edge.
559             __HAL_TIM_SET_CAPTUREPOLARITY(htim, TIM_CHANNEL_1, TIM_INPUTCHANNELPOLARITY_RISING);
560         }
561     }
562 }

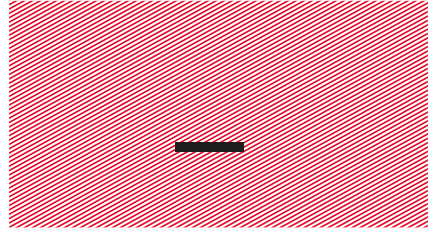
```

Automatisch kofferdeksel



A screenshot of a PuTTY terminal window titled "PuTTY (inactive)". The window displays a list of ten JSON objects, each containing a "distance" key and a numerical value. The values are: 53, 51, 54, 11, 10, 10, 10, 11, 54, 12, and 14. The text is displayed in a monospaced font on a black background.

```
{ "distance": 53 }  
{ "distance": 51 }  
{ "distance": 54 }  
{ "distance": 11 }  
{ "distance": 10 }  
{ "distance": 10 }  
{ "distance": 10 }  
{ "distance": 11 }  
{ "distance": 54 }  
{ "distance": 12 }  
{ "distance": 14 }
```



Theorieopdracht



Maak gebruik van de demo code i.v.m. UART en DMA om enkele vragen te beantwoorden.

De code heeft als naam: 'Nucleo-L476RG - DMA – UART to memory'.

Zie: "**Theorieopdracht 3 – UART RX DMA**".